



GreenCity Tech

Proposition & pilotage d'une organisation DevOps



Normandie
Web School

Master Lead Développeur Full Stack



Introduction

1 Contexte



CONTEXTE

Projet peu structuré

- Application de signalement citoyen
- Plusieurs projets (app mobile, app web, API...)
- Prototype développé rapidement
- Besoin d'adaptabilité
- Peu d'automatisation
- Déploiements manuels
- Pas de tests
- Pas de monitoring
- Pas d'organisation projet
- Communication difficile



Liam Cariou

RÔLE DE CONSULTANT DEVOPS

OBJECTIFS

Industrialiser le projet afin de :

MIEUX LIVRER

Automatiser, standardiser, fiabiliser

MIEUX EXPLOITER

Superviser, stabiliser, sécuriser

MIEUX PILOTER

Prioriser, planifier, mesurer



Organisation & méthode projet

2 Organisation d'équipe



OBJECTIF

Une équipe plus structurée afin de gagner en efficacité, en visibilité et en qualité

PILOTAGE

Product Owner

- Porte la vision produit
- Arbitre & priorise
- Protège l'équipe
- Organise

PILOTAGE

DevOps senior

- CI/CD
- Déploiement
- Environnements
- Supervision

DÉVELOPPEMENT

4 développeurs initiaux

LEAD DEVELOPER

- Coordonne l'équipe
- Répartit les tâches
- Garantit la cohérence technique

DEV MOBILE

DEV BACKEND

DEV FULL STACK

ARBITRAGES

Une équipe resserrée au lancement

- Périmètre fonctionnel encore limité
- Phase de démarrage
- Maîtrise des coûts
- Montée en compétence progressive sur les sujets DevOps
- Cohérence en passant d'un prototype à un projet plus structuré

3 Méthode & rituels



OBJECTIF

Un cadre de travail clair pour mieux coordonner l'équipe et sécuriser l'avancement

CADRE RETENU

Cycles courts

- Sprints de 2 semaines
- Backlog priorisé par le PO
- Kanban séparé pour incidents & urgences
- Décisions tracées

CADRE RETENU

Des rituels utiles

- Point dev tous les 2j : identification des blocages, réajustements
- Suivi global hebdomadaire interne avec PO, équipe DEV & DevOps
- Fin de sprint : démo client & rétrospective équipe
- Pilotage mensuel avec collectivités clientes & arbitrages

POURQUOI CE CHOIX

Structurer sans alourdir

- Équipe restreinte
- Socle fragile à structurer
- Priorités évolutives selon retours clients

DÉCISIONS PLUS RAPIDES

MOINS DE FLOU

MEILLEURE COORDINATION

LIVRAISONS PLUS FIABLES



OBJECTIF

Un lancement cadré, un budget projet lisible et maîtrisé avec la direction financière

ARBITRAGES

Des choix de lancement ciblés

- Capitalisation sur l'équipe dev du prototype
- Renfort gestion de projet & DevOps senior
- Infrastructure simple, peu coûteuse & efficace
- Pas de QA dédié ni d'équipe DevOps complète au départ

LECTURE BUDGÉTAIRE

Où va l'argent

- La **RH** est de loin le poste dominant (~95 %)
- Coûts journaliers **différenciés par profil**
- Infra & outils sobres · ordres de grandeur affinés avec le contrôle de gestion

BUDGET PROJET · ORDRES DE GRANDEUR

Phase de structuration · ~4 mois · ~120 k€

POSTE	COÛT/J	/ MOIS	PHASE
Développeurs (x4)	225 €	18 k€	72 k€
Product Owner	250 €	5 k€	20 k€
DevOps senior	300 €	6 k€	24 k€
Infrastructure (VPS, S3...)	—	0,8 k€	3,2 k€
Outils & licences	—	0,3 k€	1,2 k€
Total projet		~30 k€	~120 k€

Coûts journaliers **chargés** (moyens par profil) · le cadrage DevOps initial (~13 j) est inclus dans la ligne DevOps.



OBJECTIF

Piloter le budget projet en continu, détecter les écarts tôt et déclencher les actions correctives

PRODUCTION CONTINUE DES DONNÉES • À MI-PARCOURS

Tableau de bord budgétaire (projet)

BUDGET

120 k€

CONSOMMÉ

63,6 k€

RESTE

56,4 k€

POSTE (À DATE)	PRÉVU	RÉEL	ÉCART
RH — équipe	58	61	+5 %
Infrastructure	1,6	2,1	+31 %
Outils & licences	0,6	0,5	-17 %
Total à date (k€)	60,2	63,6	+6 %

ANALYSE DES ÉCARTS

Seuils d'alerte

- Écart < 5 % — sous contrôle
- 5 – 10 % — vigilance, analyse de cause
- > 10 % — alerte & action immédiate

PLAN D'ACTION (ÉCART / INCIDENT)

Réagir vite

- Analyse de cause (ici : +31 % infra → un VPS ajouté)
- Re-priorisation du backlog & réallocation
- Arbitrage avec le PO et la direction
- Reporting mensuel au contrôle de gestion + alerte à la direction financière si seuil rouge



03

Mise en œuvre DevOps

6 Versioning & environnements



VERSIONNING

Structuration du versioning

MAIN

Branche de production, seulement via merge depuis develop

DEVELOP

Branche d'intégration des nouvelles fonctionnalités

FEATURE/* · HOTFIX/* · DOC/*

Branches courtes destinées à un seul but



GitHub

ENVIRONNEMENTS

Environnements séparés avec leurs propres règles et leur utilité

PRODUCTION

Environnement sensible, le déploiement n'est actionné que manuellement.

PREPRODUCTION

Version candidate à la production. Accessible aux clients pour démo & recette.

DEV

Environnement de dev interne pour intégration, déploiement automatisé.



APP WEB

Portail web citoyen

Usage citoyen terrain.

APP MOBILE

App mobile

Usage citoyen terrain.

APP WEB

Interface admin

Suivi municipal, traitement des incidents.

API

API partenaires

Récupération de données, échanges externes, extensions futures.

BASE DE DONNÉES

Base de données métier

- Toutes les applications exploitent les mêmes données
- Les bases de données de chaque environnement sont isolées

STOCKAGE

Stockage de documents ou photos

- Bucket S3
- Stockage accessible par toutes les apps
- Coût à prendre en compte
- Maîtrise de l'espace disque des serveurs

CONTENEURS

Docker

- Toutes les applications sont dockerisées, sauf l'application mobile
- Déploiement plus simple, unifié et automatisé avec Docker Compose
- Images versionnées afin de faciliter le rollback



SYSTÈME

Netdata

Santé des VPS, CPU, mémoire, disque, réseau, disponibilité.



EXTERNE

UptimeRobot

Disponibilité réelle depuis l'extérieur, temps de réponse.



APPLICATIF

Sentry

Remontées d'erreurs applicatives, exceptions, fréquence.



DIAGNOSTIC

Logs structurés

Horodatage, niveaux, suivi par ID, suivi interservices.

CADRE RETENU

Supervision globale & réaliste

- Netdata sur un VPS séparé (ne tombe pas avec les apps)
- UptimeRobot pour la disponibilité vue client
- Sentry + logs pour retracer les erreurs
- Alertes en temps réel

INDICATEURS SUIVIS

Anomalies détectées via

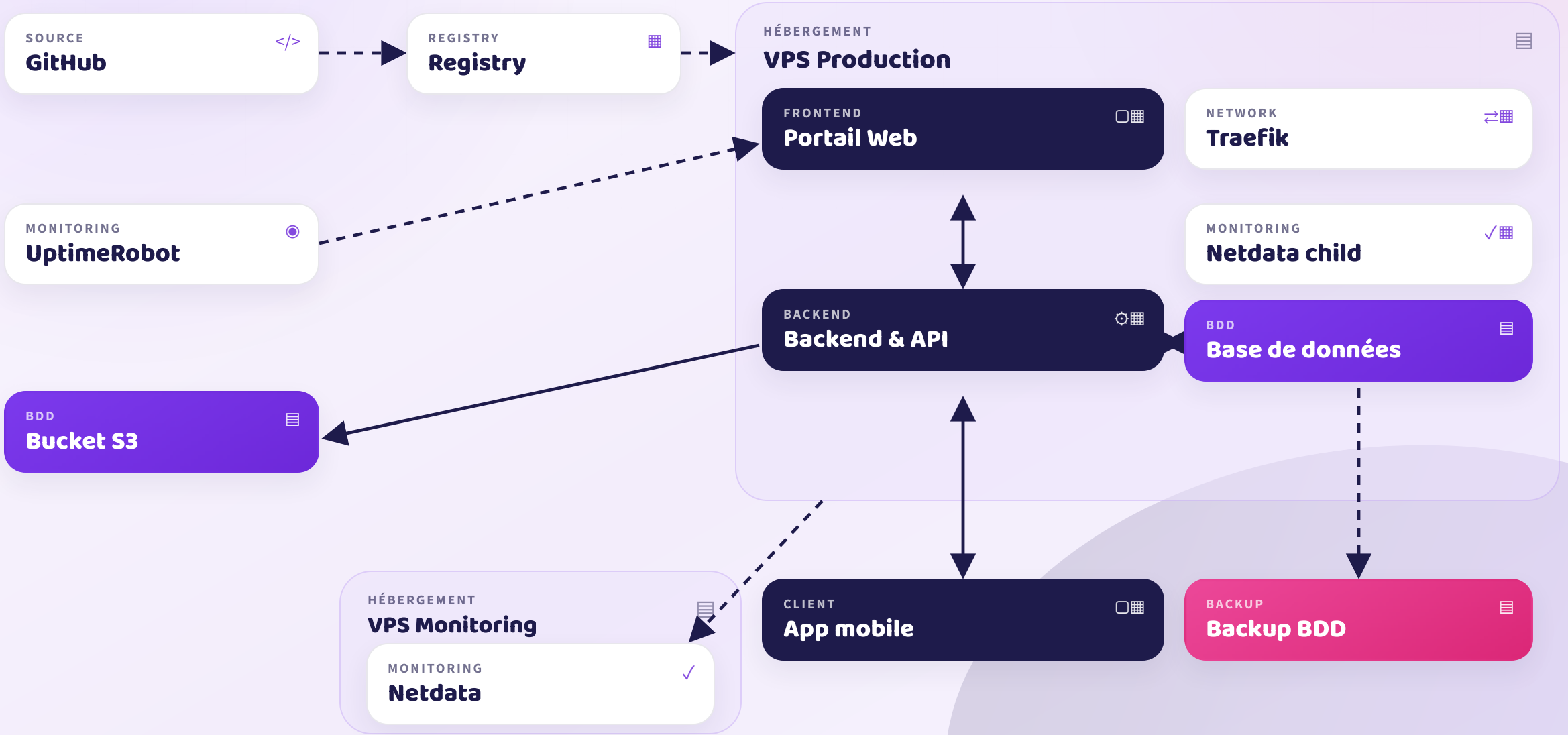
- Disponibilité des services
- Temps de réponse
- Taux d'erreurs applicatives
- Charge serveur (CPU/RAM/disque)

GESTION DES INCIDENTS

Un processus clair

- 1 Détection — alerte automatique
- 2 Qualification & priorité
- 3 Résolution (astreinte)
- 4 Communication PO / client
- 5 Post-mortem (REX) & correctif

9 Architecture déploiement





STAGE 01

Contrôles rapides

6

lint

Contrôle du code.

typecheck

Vérification du typage.

app_tests

Tests unitaires et d'intégration.

dependency_audit

Vérification des dépendances.

image_scan

Scan Trivy de l'image Docker.

accessibility_check

Vérification rapide de l'accessibilité.

STAGE 02

Build

2

app_build

Compilation de l'application.

docker_build

Construction de l'image Docker.

STAGE 03

Tests avancés

1

e2e_tests

Tests E2E sur les parcours critiques.

STAGE 04

Déploiement

4

deploy_dev

Si branche « develop ».

deploy_preprod

Si release candidate.

▶ deploy_prod

Si branche « main ».

MANUEL

smoke_check

Contrôle rapide après déploiement.

ARBITRAGES

Arbitrages CI/CD

- Production sous validation manuelle
- Secrets gérés via GitHub Secrets
- Tests E2E lancés uniquement sur branches develop & main
- Application mobile plus difficile à automatiser
- Contrôles à renforcer si le projet grandit (SonarQube, déploiement Cloud ...)



Qualité, conformité, sécurité



QUALITÉ DU CODE

Critères qualité

Ces critères permettent de définir quand une fonctionnalité est terminée :

- Code relu & fonctionnel
- Tests exécutés
- Documentation à jour
- Sécurité vérifiée
- Critères de recette validés
- Déployable
- Revue de code avec le Lead Developer

Ces standards permettent d'avancer dans la même direction au sein de l'équipe.

SÉCURITÉ

Sécurité des applications

La sécurité doit être intégrée dès le développement et le déploiement.

- Audit dépendances
- Scan vulnérabilités avec Trivy
- Secrets & variables protégées
- Bonnes pratiques OWASP
- Revues de code
- Veille des failles connues

CONFORMITÉ

Contexte collectivités

Le respect du RGPD, des normes d'accessibilité est déjà nécessaire, mais encore plus dans un contexte de travail avec des collectivités.

- Respect du RGPD et politique de confidentialité conforme
- Sécurité applicative & infrastructure pour éviter les intrusions
- Prise en compte du RGAA
- Conformité stores pour l'application mobile



TESTS WEB

Couverture globale

Unitaires, intégration sur la logique métier, E2E sur les parcours critiques.

TESTS MOBILES

Progressifs

Tests unitaires, build de validation, suivi des crashes, recette manuelle avant diffusion.

ACCESSIBILITÉ

Auto + humain (RGAA)

Scans automatiques + audit manuel RGAA — obligation pour les collectivités clientes.

PROTOCOLES DE TESTS EN SITUATION

Valider dans des conditions réelles

- Recette scénarisée en préproduction iso-prod
- Tests de charge & performance sous trafic simulé
- Tests sur devices réels + jeux de données réalistes
- Campagne de validation avant chaque release

CONFORMITÉ AUX CANAUX DE DISTRIBUTION

Publier sans blocage

Mobile (stores)

- Pre-launch report Play Console, bêta TestFlight, guidelines Apple/Google, signatures & certificats

Web

- Performance, sécurité, RGAA — checklist de conformité avant publication



05

Gestion produit et relation client



ÉTAPE 1

Retour terrain

ÉTAPE 2

Cadrage produit

ÉTAPE 3

Analyse d'impact

ÉTAPE 4

Arbitrage interne

ÉTAPE 5

Roadmap, report ou refus

CRITÈRES DE PRIORISATION

Piloter les possibles évolutions

- Valeur métier pour plusieurs collectivités
- Quantifier l'effort nécessaire
- Prévoir les risques (sécurité, RGPD ...)
- Capacité réelle technique de l'équipe
- Quantifier le coût financier et en performances

ARBITRAGE

Une bonne idée doit être cadrée

Si le besoin est trop spécifique, trop coûteux ou trop risqué, il est préférable de recadrer le besoin, le reporter ou ne pas l'intégrer.

DEMANDE CLIENT 1

Cartographie en temps réel

- Intérêt des collectivités
- Gros impact performances
- Communication temps réel via WebSocket

DEMANDE CLIENT 2

Photos HD

- Signalement renforcé
- Stockage bucket S3
- Compression
- Attention au RGPD

DEMANDE CLIENT 3

Chatbot citoyen

- Cadrage fort nécessaire
- Coûts importants
- Effet de mode IA
- Attention au RGPD



OBJECTIF

Une relation client structurée pour donner de la visibilité, valider régulièrement et ajuster la roadmap

RELATION CLIENT

Un point d'entrée unique

- Le Product Owner centralise les échanges
- Pas de sollicitations directes des développeurs
- Décisions et retours tracés

SUIVI

Des points réguliers avec les collectivités

- Point de suivi avec les collectivités pilotes
- Démonstration à chaque fin de cycle important
- Comité de pilotage mensuel pour les arbitrages

VALIDATION

Une préproduction utilisée avant livraison

- Validation explicite en préproduction
- Recueil des retours avant généralisation
- Réduction des incompréhensions et des écarts

SATISFACTION

Quelques indicateurs simples

- Satisfaction des collectivités
- Respect des délais annoncés
- Avancement de la roadmap
- Incidents / retours bloquants
- Budget ou charge consommés



Inclusion et management



FATIGUE COGNITIVE

Une organisation claire

- Réunions courtes, préparées, ciblées avec les bons interlocuteurs
- Formalisation à l'écrit de chaque décision & suivi asynchrone
- Canaux de communication définis. Les demandes client passent uniquement par le Product Owner.
- Découpage des tâches & roadmap claire

HANDICAP

Matériel adapté

Fournir le matériel adapté aux besoins du salarié (écrans, souris ergonomique, chaise confortable ...), outils accessibles, configurables.

EFFICACITÉ COLLECTIVE

Indicateurs

Suivi du moral, de la motivation et de la vitesse globale de l'équipe. Lecture collective des indicateurs afin d'éviter toute discrimination individuelle.

PRINCIPE

L'inclusion améliore aussi l'organisation

- Réunions plus courtes et préparées
- Suivi du projet plus facile
- Réduction des interruptions dans la production
- Plus de temps à travailler sur la vraie valeur

MANAGEMENT

Protéger la charge et le cadre de travail

Le Product Owner filtre les demandes clients, le management protège la charge de travail et les conditions de réalisation.



OBJECTIF

GreenCity Tech peut passer d'un prototype fragile à une plateforme multi-collectivités plus fiable, plus lisible et plus soutenable.

MIEUX LIVRER

**Automatiser, standardiser,
fiabiliser**

- Équipe structurée
- Rôles clarifiés
- Rituels projets définis
- Versioning cadré
- Pipeline CI/CD mis en place

MIEUX EXPLOITER

Superviser, stabiliser, sécuriser

- Environnements séparés
- Supervision mise en place
- Déploiements plus fiables
- Logs structurés
- Sécurité renforcée

MIEUX PILOTER

Prioriser, planifier, mesurer

- Interface Product Owner
- Canaux de communication
- Demandes qualifiées
- Arbitrages formalisés
- Roadmap partagée



GreenCity Tech

MERCI !



**Normandie
Web School**

Master Lead Développeur Full Stack